

Relatório do Projeto – Compilador de Markdown Simplificado (Markos)

Curso: CET 9729 Tecnologias e Programação de Sistemas de Informação

UFCD: Programação em Python

Centro: IEFP

Formador: João Galamba

Aluno: Guilherme Almeida

Data: 8 de Março de 2025

Relatório do Projeto – Compilador de Markdown Simplificado (Markos)	1
2.1 Introdução e Objectivos	3
2.2 Desenho e Estrutura	4
Arquitetura do software	4
Estrutura geral do funcionamento	7
Diagrama simplificado da arquitetura	8
Diagrama de classes	9
Diagrama de estados	10
2.3 Implementação	11
Ambiente de desenvolvimento	11
Bibliotecas utilizadas	11
Interface da linha de comandos	12
Exemplos de conversão Markdown para HTML	13
Funcionalidades implementadas	14
Funcionalidades não implementadas	14
2.4 Conclusão	15
Bibliografia / Webgrafia	15

2.1 Introdução e Objectivos

Este projeto foi desenvolvido no âmbito do módulo de Programação em Python e teve como principal objetivo criar um programa capaz de converter documentos escritos em **Markdown Simplificado** para **HTML**.

Markdown é uma linguagem de marcação muito utilizada atualmente para escrever documentação, páginas web simples e ficheiros de texto estruturados. A sua principal vantagem é permitir escrever texto formatado de forma simples, sem ser necessário utilizar diretamente código HTML.

Neste projeto foi desenvolvido um programa chamado **Markos**, que funciona como um pequeno compilador. O programa recebe um ficheiro escrito em Markdown e converte automaticamente esse conteúdo para HTML.

O desenvolvimento do projeto foi dividido em duas partes.

Na primeira parte, foram seguidas as gravações disponibilizadas pelo formador, onde foi implementada a estrutura base do programa e algumas funcionalidades fundamentais da linguagem Markdown.

Na segunda parte, foram adicionadas novas funcionalidades ao programa, de forma a suportar mais elementos da linguagem, como listas numeradas, formatação de texto (negrito e itálico), ligações e imagens.

Os principais objetivos deste projeto foram:

- Aprender a organizar um programa Python em vários módulos
- Praticar a leitura e processamento de ficheiros de texto
- Implementar um pequeno sistema capaz de interpretar uma linguagem simples
- Converter elementos Markdown para HTML automaticamente

Este projeto também permitiu compreender melhor o funcionamento básico de um **compilador simples**, que analisa um texto de entrada e produz um novo documento como resultado.

2.2 Desenho e Estrutura

Arquitetura do software

O programa foi desenvolvido utilizando vários ficheiros Python, de forma a separar as diferentes partes do sistema. Esta organização ajuda a tornar o código mais claro e mais fácil de manter.

Os principais componentes do programa são:

Ficheiro	Descrição
markos.py	Programa principal que executa o compilador
markdown_compiler1.py	Responsável por analisar o texto Markdown
html_backend.py	Responsável por gerar o código HTML
markdown_list.py	Contém classes utilizadas para representar listas
utils.py	Contém funções auxiliares utilizadas pelo programa

O ficheiro **markos.py** é o ponto de entrada do programa. É responsável por ler os argumentos da linha de comandos e iniciar o processo de compilação.

```
def main():
    args = docopt(dedent(doc))
    style_sheet = args['--style-sheet']
    pretty_print = args['--pretty']

    try:
        in_file = from_file_or_stdin(args['INPUT_FILE'])
        out_file = to_file_or_stdout(args['OUTPUT_FILE'])
        backend = HTMLBackend(out_file, style_sheet, pretty_print)

        with in_file, out_file, closing(backend):
            mkd_compiler = MarkdownCompiler(backend)
            mkd_compiler.compile(in_file.read())

    except FileNotFoundError as ex:
        print(f"File not found: {ex.filename}", file = sys.stderr)
        sys.exit(2)

    except PermissionError as ex:
        print(f"Invalid permissions to access file: {ex.filename}", file = sys.stderr)
        sys.exit(13)

    except Exception as ex:
        print(f"An error has occurred:\n{ex.args}\n\n", file = sys.stderr)
        raise ex

#:
if __name__ == '__main__':
    main()
#:
```

Figura 1 – Código do ficheiro principal markos.py

O módulo **markdown_compiler1.py** analisa o texto do ficheiro Markdown e identifica os diferentes elementos da linguagem, como cabeçalhos, listas ou parágrafos.

```
def compile(self, in_: TextIO | str):
    if isinstance(in_, str):
        in_ = StringIO(in_)

    backend = self._backend
    title = self._read_title(in_)
    backend.open_document(title)

    CompilerState = Enum('CompilerState', 'OUTSIDE INSIDE_PAR NEW_LIST')
    state = CompilerState.OUTSIDE

    while line := in_.readline():
        #we can't use for loop here because
        line = line[:-1]
        #we want to be able to rewind in_

        if state is CompilerState.OUTSIDE and self._is_heading_line(line):
            self._new_heading(line)

        elif state is CompilerState.OUTSIDE and matches(self.LIST_ITEM_LINE, line):
            rewind_one_line(in_, line)
            self._compile_list(in_)
            state = CompilerState.NEW_LIST

        elif state is CompilerState.OUTSIDE and matches(self.ORDERED_LIST_ITEM_LINE, line):
            rewind_one_line(in_, line)
            self._compile_ordered_list(in_)
            state = CompilerState.NEW_LIST
```

Figura 2 – Implementação da máquina de estados principal do compilador Markdown, responsável por analisar cada linha do documento e determinar se

corresponde a um cabeçalho, parágrafo ou lista.

O módulo `html_backend.py` recebe os elementos identificados pelo compilador e gera o código HTML correspondente.

```
class HTMLBackend(MarkdownBackend):
    def __init__(self, out: TextIO, style_sheet = '', pretty_print = False):
        self._storage = out
        self._out = StringIO()
        self._style_sheet = style_sheet
        self._pretty_print = pretty_print
        #:

    def close(self):
        self._storage.write(
            prettify_html(self._out.getvalue()) if self._pretty_print else
            self._out.getvalue()
        )
        self._out.close()
        #:

    @override
    def open_document(self, title=''):
        out = self._out
        out.write("""<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta http-equiv="X-UA-Compatible" content="IE=edge">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
""")
        .replace('\n', ' ')
        if title:
```

Figura 3 – Implementação do backend HTML responsável por gerar a estrutura do documento HTML a partir dos elementos Markdown processados pelo compilador.

O módulo `markdown_list.py` é utilizado para representar estruturas de listas dentro do documento.

```
class MarkdownList(list['ListItem']):
    def __init__(self, iterable: Iterable['ListItem'] = ()):
        super().__init__(iterable)
        self.with_paragraphs = False
        #:

    def add_new_list_item(self, initial_inner_elem: 'ListItemInnerElem') -> 'ListItem':
        curr_list_item = ListItem()
        curr_list_item.append(initial_inner_elem)
        self.append(curr_list_item)
        return curr_list_item
    #:

class ListItem(list['ListItemInnerElem']):
    def add_text_line(self, line: str):
        last_inner_elem = self[-1]

        if isinstance(last_inner_elem, ListItemBlock):
            last_inner_elem.add_line(line)

        elif isinstance(last_inner_elem, ListItemHeading):
            self.append(ListItemBlock(line))
```

Figura 4 – Estruturas de dados utilizadas para representar listas Markdown durante o processo de compilação, incluindo os itens da lista e os seus elementos internos.

Por fim, o ficheiro **utils.py** contém várias funções auxiliares que ajudam no processamento do texto.

```
__all__ = [
    'prettyfy_html',
    'rewind_one_line',
    'from_file_or_stdin',
    'to_file_or_stdout',
    'matches',
    'count_consec'
]

def prettyfy_html(html_code: str | TextIO, indent = 2) -> str:
    soup = BeautifulSoup(html_code, features = 'html.parser')
    return soup.prettify(formatter = HTMLFormatter(indent = indent))
#:

def rewind_one_line(in_: TextIO, line: str):
    n_chars = len(line.encode()) + 1 # '+ 1' accounts for the removed '\n'
    in_.seek(in_.tell() - n_chars, 0)
#:

def from_file_or_stdin(file_path: str | None) -> TextIO:
    return open(file_path, 'rt') if file_path else sys.stdin
#:

def to_file_or_stdout(file_path: str | None) -> TextIO:
    return open(file_path, 'wt') if file_path else sys.stdout
#:

def matches(pattern: re.Pattern, line: str) -> bool:
    return bool(pattern.fullmatch(line))
#:

def count_consec(txt: str, char: str, start_pos: int = 0):
    count = 0
```

Figura 5 – Ficheiro Utils

Estrutura geral do funcionamento

O funcionamento geral do programa pode ser descrito da seguinte forma:

1. O utilizador executa o programa através da linha de comandos.
 2. O programa recebe o ficheiro Markdown de entrada.
 3. O compilador lê o ficheiro linha a linha.
 4. O texto é analisado para identificar elementos da linguagem Markdown.
 5. Os elementos encontrados são convertidos para HTML.
 6. O programa gera um ficheiro HTML final.
-

Diagrama simplificado da arquitetura

Utilizador

|

v

markos.py

|

v

markdown_compiler1.py

|

v

html_backend.py

|

v

Ficheiro HTML

Este esquema mostra como os diferentes módulos do programa interagem entre si.

Diagrama de classes

A classe **MarkdownList** representa uma lista de elementos, enquanto a classe **ListItem** representa cada item individual dentro da lista.

Estas classes permitem organizar melhor os dados e facilitar a geração do HTML correspondente.

Diagrama de estados

Início

|

v

Ler ficheiro Markdown

|

v

Processar linhas do documento

|

v

Identificar elementos Markdown

|

v

Converter para HTML

|

v

Gerar ficheiro final

|

v

Fim

Este diagrama mostra as diferentes fases do processamento do documento.

2.3 Implementação

Ambiente de desenvolvimento

O projeto foi desenvolvido utilizando a linguagem **Python 3**.

Ferramentas utilizadas:

- Visual Studio Code
- Python 3
- Terminal do sistema operativo

O programa foi desenvolvido e testado num computador com **Windows**.

Bibliotecas utilizadas

Biblioteca	Função
docopt	Permite ler argumentos da linha de comandos
BeautifulSoup	Utilizada para formatar o HTML gerado
p	

A biblioteca **docopt** facilita a criação de interfaces de linha de comandos, permitindo interpretar os argumentos fornecidos pelo utilizador.

A biblioteca **BeautifulSoup** foi utilizada para melhorar a formatação do HTML final.

Interface da linha de comandos

O programa pode ser executado através da seguinte instrução:

```
python markos.py [-s STYLE_SHEET] [-p] [INPUT_FILE] [OUTPUT_FILE]
```

Descrição dos parâmetros:

INPUT_FILE – Ficheiro Markdown que será convertido

OUTPUT_FILE – Ficheiro HTML que será gerado

-p – Ativa a formatação automática do HTML

-s – Permite adicionar um ficheiro CSS

```
(.venv) C:\Users\guima\Desktop\CET_9729\Python\Projeto_Markdown\src>python markos.py -p tests/test0.md
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8"/>
    <meta content="IE=edge" http-equiv="X-UA-Compatible"/>
    <meta content="width=device-width, initial-scale=1.0" name="viewport"/>
    <title>
      Teste 1
    </title>
  </head>
  <body>
    <h1>
      Parte 1
    </h1>
    <p>
```

Figura 6 – Execução do programa no terminal

Exemplos de conversão Markdown para HTML

Markdown**HTML**

Título

```
<h1>Título</h1>
```

Secção

```
<h2>Secção</h2>
```

texto

```
<strong>texto</strong>
```

texto

```
<em>texto</em>
```

- Item

```
<li>Item</li>
```

Google

```
<a href="https://google.com">Google</a>
```

Funcionalidades implementadas

Durante o desenvolvimento do projeto foram implementadas as seguintes funcionalidades:

- Título do documento
- Cabeçalhos (#, ##, ###)
- Parágrafos
- Listas não numeradas
- Listas numeradas
- Texto em negrito
- Texto em itálico
- Ligações (links)
- Imagens

Estas funcionalidades permitem criar documentos Markdown relativamente completos.

Funcionalidades não implementadas

O enunciado do projeto indicava também uma funcionalidade opcional:

Desenvolvimento de um **backend para LaTeX**.

Esta funcionalidade não foi implementada neste projeto.

2.4 Conclusão

A realização deste projeto permitiu aplicar vários conhecimentos adquiridos durante o módulo de programação em Python.

Durante o desenvolvimento foi possível trabalhar com leitura de ficheiros, manipulação de texto e organização de código em diferentes módulos.

Também foi possível compreender melhor o funcionamento de um sistema que interpreta uma linguagem de marcação e gera um documento numa linguagem diferente.

O programa desenvolvido consegue converter documentos escritos em Markdown Simplificado para HTML, suportando vários elementos da linguagem como cabeçalhos, listas e formatação de texto.

Apesar de a funcionalidade extra de geração para LaTeX não ter sido implementada, o projeto cumpre os principais objetivos definidos no enunciado.

Este projeto foi uma boa oportunidade para praticar programação em Python e compreender melhor como funcionam ferramentas de conversão de documentos.

Bibliografia / Webgrafia

<https://commonmark.org>

<https://daringfireball.net/projects/markdown/>

<https://github.com/adam-p/markdown-here/wiki/Markdown-Cheatsheet>